

Yes. Below is a **real AWS Data Engineer Interview** (Investment Banking style) from **basic to advanced**. The answers are written as if you are speaking in an interview (around **2-3 minutes each**), not textbook definitions.

AWS DATA ENGINEER INTERVIEW

(Part-1)

Basic to Advanced (Q1-Q20)

Q1. Tell me about your AWS Data Engineering project.

Answer

"I worked on an investment banking project where our objective was to build a scalable data platform on AWS to process trade transactions coming from multiple upstream applications.

The data was generated by different trading applications and landed into Amazon S3 in CSV, JSON, and Parquet formats. We used AWS Glue Crawlers to scan the files and update the AWS Glue Data Catalog.

AWS Glue ETL jobs were responsible for data cleansing, removing duplicates, applying business transformations, converting CSV files into Parquet format, and partitioning the data.

For near real-time file arrivals, Amazon S3 event notifications triggered AWS Lambda functions. Lambda validated the incoming files and initiated Glue workflows.

Processed data was stored back into S3 in a curated layer.

Business users queried the data using Amazon Athena for ad-hoc analysis, while reporting teams loaded aggregated data into Amazon Redshift for dashboards.

CloudWatch monitored Glue jobs, Lambda executions, and system health. IAM roles controlled access to S3, Glue, Athena, Redshift, and CloudWatch.

The complete pipeline was highly automated and supported both full load and incremental load processing."

Q2. Explain your complete AWS architecture.

Answer

"Our architecture consisted of multiple AWS services working together.

Source systems generated transaction files.

↓

Files landed in Amazon S3 Raw Bucket.

↓

S3 Event Notification triggered Lambda.

↓

Lambda validated filename, schema, and duplicate files.

↓

Lambda started AWS Glue Workflow.

↓

Glue Crawler updated Data Catalog.

↓

Glue ETL Job transformed data.

↓

Data stored into Curated S3 Bucket as Parquet.

↓

Athena queried S3 directly.

↓

Important business tables loaded into Amazon Redshift.

↓

CloudWatch monitored the entire workflow.

IAM roles provided secure access between every AWS service."

Q3. Why did you use Amazon S3?

Answer

"Amazon S3 was our enterprise data lake.

We selected S3 because it offers virtually unlimited storage, high durability of eleven nines, high availability, and low storage cost.

It supports structured, semi-structured, and unstructured data.

It integrates easily with Glue, Athena, EMR, Lambda, Redshift Spectrum, and many other AWS analytics services.

Another advantage was lifecycle management. Older data automatically moved from Standard storage to Glacier for cost optimisation."

Q4. Explain S3 Storage Classes.

Answer

"We used different storage classes depending on data usage.

S3 Standard was used for frequently accessed current data.

Standard IA stored historical files accessed occasionally.

One Zone IA was used for non-critical backup data.

Glacier Instant Retrieval stored archived reports requiring occasional access.

Glacier Flexible Retrieval stored monthly archives.

Glacier Deep Archive stored compliance data retained for several years.

Lifecycle policies automatically moved objects between these storage classes."

Q5. What is an S3 Bucket Policy?

Answer

"A Bucket Policy controls permissions at the bucket level.

For example,

Only Glue IAM Role can read raw bucket.

Only Lambda Role can upload validation reports.

Only Redshift Role can access curated bucket.

Everyone else receives Access Denied.

Bucket Policies are resource-based policies."

Q6. Difference between IAM Role and IAM User?

Answer

IAM User

- Permanent identity
- Login credentials
- Used by humans

IAM Role

- Temporary credentials
- Assumed by AWS Services
- No password

Example:

Glue Job assumes Glue IAM Role.

Lambda assumes Lambda Role.

EC2 assumes EC2 Role.

No access keys are stored inside code.

This improves security significantly."

Q7. Explain IAM Policy.

Answer

"IAM Policy is a JSON document defining Allow or Deny permissions.

For example,

Glue Role permissions:

- Read Raw S3
- Write Curated S3
- Update Glue Catalog
- Create CloudWatch Logs

Following the Principle of Least Privilege, we granted only the permissions required for each service."

Q8. Explain AWS Glue.

Answer

"AWS Glue is a serverless ETL service.

Instead of managing Spark clusters ourselves, AWS automatically provisions infrastructure.

Glue Jobs performed

- Data cleansing
- Null handling
- Duplicate removal
- Type conversion
- Joins
- Aggregations
- Partition creation
- Writing Parquet

Glue reduced infrastructure management while providing scalable Spark execution."

Q9. Explain Glue Job Flow.

Answer

"The Glue job started after file validation.

Steps:

Read CSV from S3

↓

Apply schema

↓

Validate records

↓

Remove duplicates

↓

Handle null values

↓

Business transformations

↓

Convert CSV to Parquet

↓

Partition by TradeDate

↓

Store in Curated Bucket

↓

Update Glue Catalog

↓

Send CloudWatch Metrics"

Q10. What is Glue Crawler?

Answer

"Glue Crawler scans S3 folders.

It automatically identifies

- File format
- Column names
- Datatypes
- Partitions

Crawler creates metadata inside Glue Data Catalog.

Athena queries metadata instead of scanning folders manually.

Whenever new partitions arrive, crawler updates the catalog automatically."

Q11. What is Glue Data Catalog?

Answer

"The Glue Data Catalog is a centralized metadata repository.

It stores

Database Name

Table Name

Columns

Datatypes

Partitions

S3 Location

Athena, EMR, Glue, and Redshift Spectrum all use the same metadata."

Q12. Difference between Crawler and Catalog?

Answer

Crawler

Discovers metadata automatically.

Catalog

Stores discovered metadata.

Crawler updates Catalog.

Athena reads Catalog.

Q13. Explain AWS Lambda in your project.

Answer

"Lambda handled lightweight event-driven processing.

Whenever a new file arrived in S3,

Lambda

validated filename,

validated extension,

checked duplicate files,

verified file size,

performed basic schema validation,

and started the Glue Workflow.

Lambda completed within a few seconds without maintaining any servers."

Q14. Explain CloudWatch.

Answer

"We used CloudWatch for monitoring AWS services.

CloudWatch collected

Glue Job status

Lambda logs

S3 Events

CPU metrics

Memory metrics

Execution duration

Failure count

Custom metrics

Whenever a Glue Job failed,

CloudWatch Alarm notified the support team through SNS email."

Q15. What logs did you monitor?

Answer

"We monitored

Glue Driver Logs

Glue Executor Logs

Lambda Logs

CloudWatch Logs

S3 Access Logs

Athena Query History

Redshift Query Logs

CloudTrail Audit Logs

These logs helped identify failures quickly."

Q16. Explain Full Load.

Answer

"Full Load means loading the complete source dataset every execution.

Example:

Trade table contains 100 million records.

Every night,

Delete target table.

Load all 100 million records again.

Advantages

Simple implementation.

Disadvantages

Slow.

Expensive.

Consumes more resources."

Q17. Explain Incremental Load.

Answer

"Incremental Load processes only newly added or modified records.

Suppose yesterday we loaded

100 million records.

Today only

20,000 new trades arrived.

Instead of loading all data,

Glue processes only those 20,000 records.

Incremental loading significantly reduces execution time and AWS cost."

Q18. How did you implement Incremental Load?

Answer

"We maintained watermark columns such as

LastUpdatedTime

CreatedDate

TradeTimestamp

Glue Job first read the maximum processed timestamp.

Then source query extracted only records greater than the watermark.

After successful completion,

the watermark table was updated for the next execution."

Q19. Explain Partitioning.

Answer

"We partitioned data using BusinessDate.

Instead of storing

Trades.parquet

we stored

TradeDate=2026-07-28/

TradeDate=2026-07-29/

TradeDate=2026-07-30/

Athena scans only the required partition.

Instead of scanning 5 TB,

Athena scans only 30 GB.

This improves performance and reduces query cost."

Q20. What is your Repartitioning Policy?

Answer

"In Spark-based Glue jobs, repartitioning was done to optimise output file sizes and improve parallelism.

For very large datasets, many tiny files can hurt Athena and downstream performance. Before writing the output, we used `repartition()` on business keys such as TradeDate or Region to create an appropriate number of partitions based on the data volume.

For example, after processing 500 GB of trade data, we repartitioned it into around 200 partitions so each output file was roughly 128–256 MB. This reduced the small-file problem, improved parallel processing, and made Athena and EMR queries more efficient.

When reducing the number of partitions after filtering or aggregation, we used `coalesce()` to avoid unnecessary data shuffling."

Topics Covered

- ✓ S3
- ✓ IAM
- ✓ IAM Roles
- ✓ IAM Policies
- ✓ AWS Glue
- ✓ Glue Job
- ✓ Glue Crawler
- ✓ Glue Data Catalog
- ✓ Lambda
- ✓ CloudWatch
- ✓ Full Load
- ✓ Incremental Load
- ✓ Partitioning
- ✓ Repartitioning Policy

The next set (Q21-Q50) can cover:

- Amazon Athena (advanced)

- Amazon Redshift & Redshift Spectrum
- Amazon EMR
- Snowflake architecture and Snowpipe
- File formats (CSV, JSON, Avro, ORC, Parquet)
- Slowly Changing Dimensions (SCD Types 1 & 2)
- CDC (Change Data Capture)
- Glue Workflows and Triggers
- Performance tuning and cost optimisation
- Real production troubleshooting scenarios in investment banking
- Scenario-based questions with 2–3 minute interview answers.

Perfect. Below is **Part-2 (Q21-Q50)**. These are **real AWS Data Engineer interview questions** with answers suitable for **2–3 minute interview responses**, especially for **Investment Banking / Financial Services** projects.

AWS DATA ENGINEER INTERVIEW

Part-2 (Q21-Q50)

Q21. Why did you use Amazon Athena?

Answer

"In our project, business analysts and support teams needed to query data stored in Amazon S3 without moving it into a database. We used Amazon Athena because it is a serverless query service based on Presto/Trino that allows us to run standard SQL directly on files in S3.

Our Glue Crawler created metadata in the Glue Data Catalog, and Athena used that catalog to understand the schema.

We stored data in Parquet format with partitioning by TradeDate. Because Athena scanned only the required partitions and selected columns, query performance was much faster and the cost was lower. It was mainly used for ad-hoc analysis, data validation, reconciliation, and troubleshooting."

Q22. Why Parquet instead of CSV?

Answer

"We initially received CSV files from source systems, but after processing we converted them to Parquet.

CSV is row-based, larger in size, and requires scanning the entire file.

Parquet is a columnar storage format that supports compression and column pruning.

For example, if a table has 100 columns but the query needs only 5 columns, Athena reads only those 5 columns from a Parquet file. This reduces I/O, improves query performance, and lowers Athena costs.

Because of these benefits, all curated data was stored in Parquet."

Q23. Difference between Athena and Redshift?

Answer

Athena	Redshift
Serverless	Managed Data Warehouse
Queries S3 directly	Stores data inside cluster
Pay per query	Pay for compute/storage
Best for ad-hoc queries	Best for BI dashboards
No infrastructure	Cluster management

Interview Answer

"We used Athena for ad-hoc investigation and validation because it queried S3 directly without loading data. For reporting dashboards and complex analytics with high concurrency, we loaded curated datasets into Amazon Redshift."

Q24. What is Amazon Redshift?

Answer

"Amazon Redshift is AWS's fully managed cloud data warehouse designed for analytical workloads.

After Glue transformed and cleaned the data, we loaded business-critical tables into Redshift.

Business Intelligence tools connected to Redshift to generate daily, weekly, and monthly reports.

Redshift provided high-performance SQL execution using columnar storage, compression, and massively parallel processing."

Q25. How did you load data into Redshift?

Answer

"Our process was:

Source Files

↓

S3 Raw

↓

Glue ETL

↓

Curated Parquet

↓

COPY Command

↓

Amazon Redshift

The COPY command read Parquet files directly from S3 using an IAM Role, making it much faster than inserting records individually."

Q26. What is the Redshift COPY Command?

Answer

"The COPY command is the fastest method to load large datasets into Amazon Redshift.

Instead of inserting one row at a time, COPY reads multiple files in parallel from S3.

Benefits include:

- Parallel loading
 - High throughput
 - Better performance
 - Automatic decompression
 - IAM Role integration"
-

Q27. What is Redshift Spectrum?

Answer

"Redshift Spectrum allows Redshift to query external data stored in Amazon S3 without loading it into Redshift tables.

Spectrum uses the Glue Data Catalog to understand table metadata.

We used Spectrum mainly for historical data that was queried occasionally. Frequently used data was loaded into Redshift, while historical data remained in S3."

Q28. What is Amazon EMR?

Answer

"Amazon EMR is a managed big data platform for running Apache Spark, Hadoop, Hive, HBase, Presto, and other open-source frameworks.

Compared to Glue, EMR provides greater control over the cluster, Spark configurations, and resource allocation.

We used Glue for regular ETL jobs and EMR for large-scale Spark processing involving hundreds of gigabytes or terabytes of data."

Q29. Why EMR instead of Glue?

Answer

"Glue is serverless and ideal for standard ETL.

EMR is preferred when:

- Custom Spark configurations are needed
- Long-running jobs exist
- Very large datasets are processed
- Third-party Spark libraries are required
- Full cluster control is needed

For enterprise-scale processing, EMR offers more flexibility."

Q30. Explain Snowflake.

Answer

"Snowflake is a cloud-native data warehouse that separates storage from compute.

Unlike traditional databases, storage and virtual warehouses scale independently.

We loaded curated data into Snowflake for advanced analytics, reporting, and secure data sharing across business teams."

Q31. Why Snowflake instead of Redshift?

Answer

"Snowflake provides:

- Separate storage and compute
- Auto-scaling
- Auto-suspend/resume
- Zero-copy cloning
- Time Travel
- Secure Data Sharing

Redshift is tightly integrated with AWS and is cost-effective for AWS-centric environments.

The choice depends on business and architecture requirements."

Q32. Explain Snowflake Architecture.

Answer

"Snowflake has three layers:

1. Storage Layer – stores compressed data.
2. Compute Layer – Virtual Warehouses execute queries.
3. Cloud Services Layer – authentication, metadata, optimisation, and query management.

This separation allows independent scaling of compute without affecting storage."

Q33. What is Snowpipe?

Answer

"Snowpipe is Snowflake's continuous data ingestion service.

Whenever a file arrives in cloud storage such as Amazon S3, Snowpipe automatically detects the file and loads it into Snowflake tables.

This removes the need for scheduled batch jobs and supports near real-time ingestion."

Q34. Difference between Batch and Streaming?

Answer

Batch:

- Scheduled
- Large datasets
- Hourly/Daily

Streaming:

- Continuous
- Small events
- Seconds

"In our project, end-of-day trade processing used batch jobs, while near real-time trade validation used event-driven processing with Lambda."

Q35. What file formats have you worked with?

Answer

"I have worked with:

- CSV
- JSON
- XML
- Parquet
- ORC
- Avro

Source systems mainly generated CSV and JSON. We converted the processed data into Parquet because it provides better compression and faster query performance."

Q36. Why ORC and Parquet?

Answer

"Both are columnar storage formats.

Parquet is commonly used with Spark and data lakes.

ORC is often preferred with Hive workloads.

Both provide compression, faster reads, and reduced storage compared to CSV."

Q37. What is Compression?

Answer

"Compression reduces storage size and improves query performance.

Common compression methods:

- Snappy
- Gzip

- ZSTD
- LZO

For Parquet files, we mainly used Snappy because it offers a good balance between compression ratio and decompression speed."

Q38. Explain Partition Pruning.

Answer

"If data is partitioned by TradeDate and the query filters on a specific TradeDate, Athena or Spark scans only that partition instead of the entire dataset.

For example, querying one day from a 5 TB dataset might require scanning only 40 GB.

This significantly reduces query time and cost."

Q39. Explain Small File Problem.

Answer

"Many tiny files reduce Spark performance because each file creates additional metadata and task overhead.

To solve this, we:

- Repartitioned data before writing
 - Combined small files into larger files
 - Targeted file sizes around 128–256 MB
 - Used coalesce() after filtering when appropriate"
-

Q40. Difference between Repartition and Coalesce?

Answer

Repartition

- Increases or decreases partitions

- Performs a full shuffle
- Better for balancing data

Coalesce

- Mainly reduces partitions
- Minimal shuffle
- Faster when decreasing partitions

"We used repartition before large writes and coalesce after filtering to reduce the number of output files efficiently."

Q41. What is Full Refresh?

Answer

"A full refresh deletes existing target data and reloads everything from the source."

We used this during initial data loads, major schema changes, or when data corruption required rebuilding the dataset."

Q42. What is Delta Load?

Answer

"A delta load processes only newly inserted or updated records."

It is much faster than a full refresh and was our standard approach for daily ETL jobs."

Q43. How did you identify changed records?

Answer

"We used:

- LastUpdatedTimestamp
- CreatedDate
- ModifiedDate

- Transaction ID
- Change sequence numbers

The Glue job compared these values with the stored watermark to identify new or modified records."

Q44. How did you handle duplicate records?

Answer

"We identified duplicates using business keys such as TradeID and AccountID.

Duplicates were removed using Spark's `dropDuplicates()` on the relevant columns.

We also validated duplicate file names in Lambda before starting the ETL process."

Q45. How did you handle schema changes?

Answer

"When new columns were introduced:

- Glue Crawler detected the changes
 - Glue Data Catalog was updated
 - Glue ETL jobs were modified to populate default values where needed
 - Downstream Athena and Redshift tables were validated to ensure compatibility"
-

Q46. What monitoring did you implement?

Answer

"We monitored:

- Glue job success/failure
- Lambda execution
- S3 events
- CloudWatch metrics
- Redshift query performance

- Athena query history
- EMR cluster health
- Failed file processing

CloudWatch alarms triggered Amazon SNS notifications to the support team."

Q47. What was the biggest production issue?

Answer

"One day a source system delivered files with an unexpected schema change. The Glue job failed because mandatory columns were missing.

We identified the issue through CloudWatch logs, validated the incoming file, coordinated with the source application team, updated the Glue mapping after confirmation, reran the failed job, and verified downstream reports.

To avoid similar incidents, we added schema validation in Lambda before starting the ETL workflow."

Q48. How did you optimise Glue jobs?

Answer

"We improved Glue performance by:

- Converting CSV to Parquet
 - Partitioning by TradeDate
 - Repartitioning before writing
 - Filtering data early
 - Selecting only required columns
 - Using incremental loads instead of full loads
 - Avoiding unnecessary shuffles
 - Tuning Spark worker configuration based on data volume"
-

Q49. Explain the complete daily workflow.

Answer

"Our daily pipeline was:

1. Source systems generated trade files.
2. Files landed in the S3 Raw bucket.
3. Lambda validated the files.
4. Glue Crawler updated the Data Catalog.
5. Glue ETL cleaned and transformed the data.
6. Data was converted to Parquet and partitioned.
7. Curated data was written back to S3.
8. Athena was used for validation and ad-hoc queries.
9. Aggregated data was loaded into Redshift.
10. Dashboards were refreshed.
11. CloudWatch monitored all components, and SNS sent alerts if failures occurred."

Q50. Explain your project end-to-end in 3 minutes.

Answer

"My project was an AWS-based data engineering platform for an investment banking application. Multiple upstream trading systems generated transaction files in CSV and JSON formats, which were stored in an Amazon S3 Raw bucket.

An S3 event triggered an AWS Lambda function that validated the file name, schema, size, and checked for duplicates before initiating the ETL process. AWS Glue Crawlers scanned the data and updated the Glue Data Catalog. Glue ETL jobs then cleaned the data by removing duplicates, handling null values, applying business rules, converting the files into Parquet format, and partitioning them by TradeDate.

The transformed data was written to a Curated S3 bucket. Business users queried this data using Amazon Athena for ad-hoc analysis, while reporting datasets were loaded into Amazon Redshift using the COPY command for high-performance analytics. For very large Spark workloads requiring greater control, we used Amazon EMR.

CloudWatch monitored Glue jobs, Lambda functions, and infrastructure, while SNS generated alerts for failures. IAM roles enforced secure access across all AWS services following the principle of least privilege. To optimise performance and cost, we used incremental loading, partition pruning, Parquet compression, and appropriate repartitioning strategies.

This architecture provided a scalable, secure, and cost-effective data platform capable of processing large volumes of financial data efficiently."

These 50 questions cover the core AWS Data Engineer topics. A strong next step would be a **Part-3 (Q51–Q100)** focused entirely on **real production scenarios**, including troubleshooting Glue failures, Athena query issues, S3 permission errors, IAM access problems, Redshift performance tuning, EMR optimisation, Snowflake loading issues, and investment banking data pipeline incidents, with interview-style answers.